

Design of a Hamming Distance Accelerator based on a General-purpose Processor Architecture

Damian Baraniak

Camilo Sebastian Lozoya Enriquez

October 2024

1 Introduction

This rapport concerns an assignment from the Computer Architecture and Organization course offered at the University of Twente. The objective of this assignment was to design a custom computer architecture that can perform Hamming Distance calculations more efficiently than a general-purpose processor.

2 High-level language software implementation

To understand Hamming Distance calculations, solution was implemented in a high-level language C. Number of sequences, their length in bits and seed for randomness are passed as program arguments.

Every sequence was build using blocks to fit all the bits. This came down to the balancing between size of allocated memory and speed of calculations. Making the blocks bigger meant less calculations, but could result in unnecessary allocated memory. For example representing 33 bits with `uint32_t` would result in allocating 8 bytes of memory and wasting 31 bits. After some consideration it was decided to use `uint16_t`, which is believed to keep the balance between potentially wasted memory and numbers of calculations.

To check and compare the results between different sizes of blocks, created a macro in the code allowing to change size of the block at the compile time between `uint8_t`, `uint16_t` and `uint32_t`. Due to implementation of `rand` in C it isn't possible to use `uint64_t`, given how generating sequences is done. Example of compiling the program and setting size of the block to 32 bits, python script automating compiling and running uses 16 bits as default.

```
$ gcc main.c -o hamming_distance -DBLOCK_TYPE=uint32_t
```

After conducting additional tests timing the results it is obvious that bigger blocks, while allocating more memory can compute results twice as fast compared to smaller blocks.

3 Assembly implementation

Next step was to implement the same hamming distance calculations using MIPS assembly language. Main differences consisted of static memory allocation and getting binary strings from the user.

After some consideration, it was decided to represent data using ASCII encoding, length L means number of characters in one sequence. Such solutions allows to dual approaches to get the results. The simple one is to use characters **0** and **1**, which is conceptually simple and leverages fact, that binary representation of these characters differs with one bit. Another, more memory efficient approach is to utilize entire ASCII encoding, to compare more complex sequences of bits. However this approach requires knowledge how all the characters are encoded in binary.

Unfortunately this solutions, even in more complex approach is limited. All ASCII characters use only 7 bits, most significant is always **0**. Another this is, some characters are special and difficult to use in the program.

Nevertheless using ASCII characters is simple and reliable way to store data for the Hamming distance calculations. Using one byte long chunks allows less wasted memory however more iterations are to be made.

4 Specified ISA

Based on the assembly implementation it is possible to calculate the Hamming Distance using only two types of instruction with a total of eight different instructions, four of each type, and eleven registers, reducing instruction set. Building on top of that, it is possible to define some specifications about the custom processor.

- The register file is 16 registers of 16 bits, meaning all registers can be addressed with only 4 bits.
- Data is represented as 16 bits unsigned integers.

- Only 8 instructions allows to encode them with a 3 bits opcode.
- Size of an instruction is 24 bits or 3 bytes.
- Word is 3 bytes

To avoid issue with the bus being used by fetching data and instructions at the same time. it was decided to implement the Harvard Architecture, allowing to pipeline the cycles. Using said architecture there is a separate memory of the instruction which is word addressable and has memory of 8kB, so it can be addressed with 13 bits. Data memory is byte addressable and has a memory of 65kB, meaning addressing with 16 bits. Data memory is significantly bigger than the instruction memory so the processor can calculate Hamming Distance between N strings L bytes long, while $N \cdot L \leq 2^{16}$.

There are two types of instruction in the custom ISA and they are encoded in different ways.

4.1 R-Type instruction

This type of instructions is used for operations between 2 registers, and the result is stored in another register. This type of instruction is encoded:

Opcode	Rd	Rs	Rt	Shift	Unused
3 bits	4 bits	4 bits	4 bits	1 bit	8 bits

Table 1: R-Type instruction encoding

This instruction type consists of opcode of the instruction, 3 registers addresses and a single bit for shift 1 (see 3). The datapath is the same for all these instructions. The CPU fetches the register and sets the values stored in **Rs** and **Rd** as the input of the ALU, the opcode indicates operation executed by the ALU. Output is stored in the **Rd**.

Shift 1 bit is a special field for the instruction **srl**, normally this instruction shifts logically value of the **Rs** by value of the **Rt** (actually it is $Rt \% 16$). However shifting by only one bit is a common instruction it is distinguished. If Shift 1 bit is set **Rt** will be ignored.

4.2 I-Type instruction

This type of instructions executes operations between the values of a register and an immediate value, the result is stored in another register, except for the branch instructions. This type of instruction is encoded:

Opcode	Rd	Rs	Immediate value
3 bits	4 bits	4 bits	13 bits

Table 2: I-Type instruction encoding

The I-Type instructions are divided into 2 arithmetic, 1 branch and 1 memory access instructions. The arithmetic instructions follow the same datapath as the R-Type instructions, but instead of using value of **Rt**, immediate value is used. Load instruction follows the same datapath, but the result is not stored immediately in the register destination, instead it fetches into the data memory and loads the value of the register **Rd**. The branch instruction is different, values of **Rs** and **Rd** are compared in the ALU using XOR, then signal is send to the program counter should the address of the immediate become next instruction. No value is loaded into **Rd**.

4.3 Summary of the ISA

All information about the ISA are represented in the table 3:

5 Design of a custom processor datapath

As the starting point for the datapath design, basic processor from the lectures was used, with all the modifications described in the ISA. It is 16 registers of 16 bits, instruction register of 24 bits, a PC of 12 bits and ALU with 2 16 bits inputs, 1 16 bits output, opcode input and flag signaling when the result is 0. Figure 1 shows the desing of the processor.

It is a multi-cycle processor, so the instructions can be pipelined increasing efficiency. The instruction cycle is broken into 5 stages, separated by registers: Instruction Fetch, Decode and register fetch, execution, memory access, saving data in the registers. To avoid data hazards, pass-forward policy is implemented, when detected by the compiler and encoded as a 1 in the LSB of the instruction, this instruction forwards data from the ALU

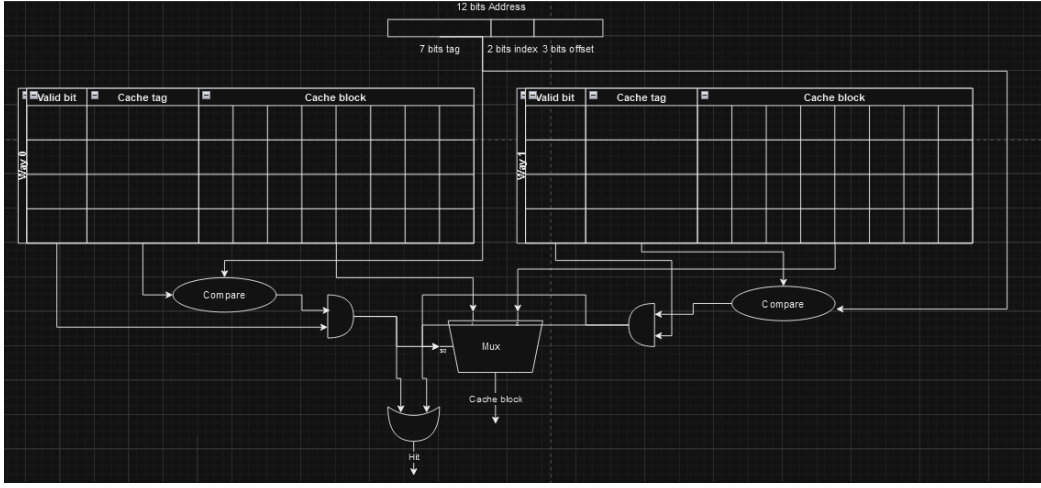


Figure 2: Cache memory design

$$\text{CPL_Stall} = 28\% \cdot 2 + 11\% \cdot (1 + 0.0665 \cdot 50) = 1.03$$

$$\text{Execution efficiency} = \frac{1}{1 + 1.03} = 49.1\%$$

6 Low-level implementation of Hamming Distance calculations

In order to calculate the Hamming distance between 2 strings of L bits, sequential logic was used. Circuit has 3 inputs: the clock signal and values for first and second binary string. Data is assumed to be 2 streams of bits that are passed consecutively to the inputs. Bits are then compared using XOR and the result is passed to the counter. Everything is updated with the positive edge of the clock. Figure 3 shows example implementation with 4-bit counter.

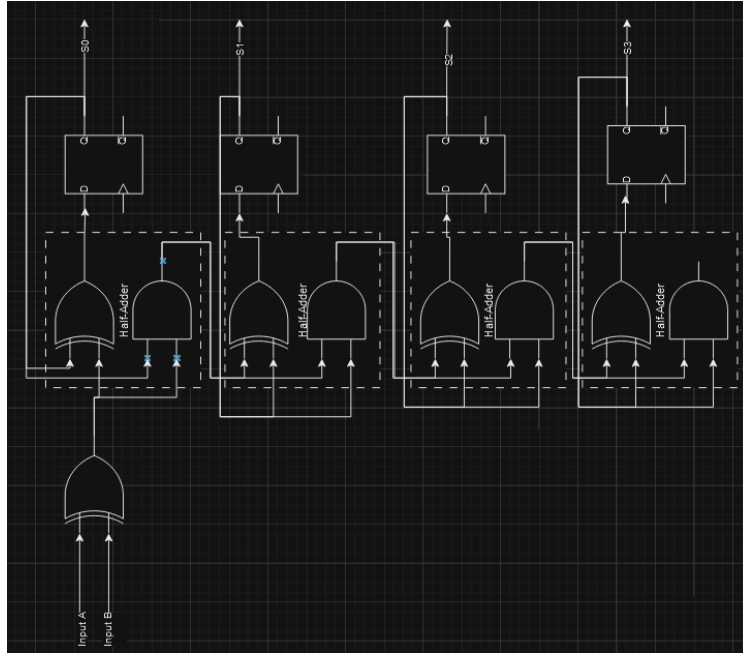


Figure 3: Logic circuit for the Hamming distance calculation

Main issue with this design is dependence on knowing length of the strings beforehand, so the sufficient counter can be used. Number of D flip-flops and half-adders needed to represent data without overflow is $\log_2(L)$. On the figure 3 there are 4 half-adders and flip-flops. Luckily expanding the circuit is simple and follows the same pattern. One of the strengths of this design is linear time complexity, it will take only $L + 1$ clock cycles to compare the strings, making it the most efficient way to calculate the Hamming distance with sequential logic.